포켓몬 150종 CNN 분류 최적화 실험 리포트

Oct 16, 2026 · @강민구

개요

CNN으로 포켓몬 150종을 분류해 **테스트 정확도 약 34~36%**(무작위로 찍을 때 0.67%의 약 53배)를 얻었다. 3차례 자동 실험에서 성능에 가장 큰 영향을 준 것은 개별 설정값이 아니라 학습량, 즉 **에포크 수**였다.

항목	내용
과제	반나절 CNN 미니 팀프로젝트. 같은 데이터셋으로 각자 실험 후 결과 비교
목표	변수를 바꿔가며 정확도와 속도가 가장 좋은 최적값 찾기
데이터셋	포켓몬 이미지 150종, 5243장, 전부 224×224 jpg, 라벨은 CSV(id, label)
분할	학습 80% 4194장 / 테스트 20% 1049장, 종류당 평균 약 35장
출발 코드	PyTorch CNN 예제 (CIFAR-10 튜토리얼 기반 conv 2층, 전날 건담 2분류에 사용)
환경	Google Colab, T4 GPU

실험 환경과 방법

예제 코드를 수정해 자동 탐색 시스템을 만들고, 한 번에 한 변수만 바꾸는 단계별 탐색으로 3차례 실험했다.

데이터 준비. 사진 zip을 드라이브에 올려 Colab에서 풀고, CSV 라벨대로 포켓몬별 폴더에 복사해 `ImageFolder`로 읽었다. 폴더 이름을 정답으로 쓰면 예제 코드를 거의 그대로 쓸 수 있어서다. 학습/테스트 분할은 시드 42로 고정해 모든 실험에서 같게 유지했다.

CPU → GPU 전환. 첫 기준 측정(3층, 224, SGD 0.001)은 1 에포크에 137초, loss 5.013, 정확도 1.0%였다. loss 5.01은 ln(150)으로, 150종을 무작위로 찍는 수준이다. 확인해 보니 CPU로 돌고 있었고, T4 GPU로 바꾸자 1 에포크가 7~10초로 약 15배 빨라졌다.

자동 탐색 시스템.

1. **FlexNet**: 층 수와 이미지 크기를 넣으면 모델을 만들고, 가짜 사진을 한 번 통과시켜 fc1 크기를 자동 계산한다. 크기를 바꿀 때마다 나던 shape 에러를 없앴다.

2. **run_experiment**: 설정 하나를 받아 학습하고 정확도, 학습 시간, 테스트 시간, 파라미터 수를 돌려준다. 학습과 테스트 시간은 따로 잰다.
3. **단계별 탐색**: 각 단계의 최고 설정을 다음 단계의 기준으로 넘긴다. 모든 조합(162번) 대신 약 13번으로 줄였다.
4. **기록**: 실험마다 시각이 붙은 CSV로 드라이브에 저장해 덮어쓰기를 막았다. 본 실험 전에 1 에포크 시험 실행(QUICK 모드)으로 에러를 확인했다.

선택 기준. 테스트 정확도가 가장 높은 설정, 같으면 학습 시간이 짧은 쪽을 골랐다.

1차 실험: 학습 설정, 이미지 크기, 층 수 (10 에포크)

2층, 224, 배치 16, Adam 0.0001이 26.7%로 가장 높았다. 출발 설정은 3층, 224, 배치 32, SGD 0.001이다.

단계	비교 값	테스트 정확도 (%)	선택
옵티마이저/학습률	SGD 0.01 / SGD 0.001 / Adam 0.001 / Adam 0.0001	21.3 / 9.0 / 22.6 / 23.0	Adam 0.0001
배치 크기	16 / 32 / 64	26.0 / 23.0 / 24.6	16
이미지 크기	224 / 128 / 64	26.0 / 25.5 / 16.5	224
층 수	2 / 3 / 4	26.7 / 26.0 / 25.5	2층

- **Adam이 초반 학습에 유리.** SGD 0.001은 10 에포크에도 9.0%에 그쳤다.
- **과적합 관찰.** Adam 0.001은 학습 loss가 0.248까지 떨어졌지만(학습 사진을 거의 외움) 테스트는 22.6%였다.
- **4층의 효율.** 4층은 2층과 정확도가 1.2%p 차이인데 파라미터는 86만 개로 2층(542만 개)의 1/6이다. 층이 깊어질수록 fc1이 작아지기 때문이다.
- **CPU 병목.** 128이 224보다 빠르지 않았다(76.9초 vs 72.6초). 모든 실험의 학습 시간이 60~77초로 비슷했다. GPU 계산보다 CPU의 사진 불러오기가 느렸다.
- **해상도 한계.** 128까지는 영향이 거의 없고 64에서 크게 떨어졌다.

2차 실험: 과적합 대응, 시드 검증, 속도 (10 에포크)

시드만 바꿔도 정확도가 최대 6.4%p 흔들려, 1차의 1~3%p 차이는 우연 범위임을 확인했다. 기준은 1차 최적 설정(2층, 224, 배치 16, Adam 0.0001)이다.

실험	결과	해석
Dropout 0 / 0.3 / 0.5	26.7 / 20.1 / 15.9%	짧은 학습에서 학습을 늦추기만 함
기본 증강 끄기 / 켜기	26.7 / 27.0%, 학습 70.8 / 80.2초	학습 loss는 오르고(1.80 → 2.42) 테스트는 소폭 상승, 과적합 완화 신호
num_workers 2 / 4	80.2 / 82.1초	효과 없음. 무료 Colab은 CPU 2개

시드 검증 (증강 켬, 시드 42·7·123)

층 수	평균 (%)	최소 ~ 최대 (%)	파라미터 수
2층	26.3	24.2 ~ 27.6	5,419,186
3층	25.1	22.4 ~ 28.8	2,250,578
4층	21.5	18.2 ~ 24.1	858,002

최고 단일 기록 28.8%(3층, 시드 42)는 운이었다. 평균으로는 2층이 가장 높고, 범위도 가장 좁아 안정적이다.

3차 실험: 증강 강도 × 에포크, BatchNorm

에포크를 10에서 20으로 늘리자 정확도가 24.2%에서 35.7%로 11.5%p 올랐다. 이날 실험 중 가장 큰 개선이다.

강한 증강은 랜덤 크롭(80~100%), 15° 회전, 밝기·대비 변화, 가우시안 노이즈를 더했다. 색상은 포켓몬을 구분하는 핵심 단서라 바꾸지 않았다. 1 에포크 학습 시간은 기본 8.4초, 강한 증강 20.7초였다.

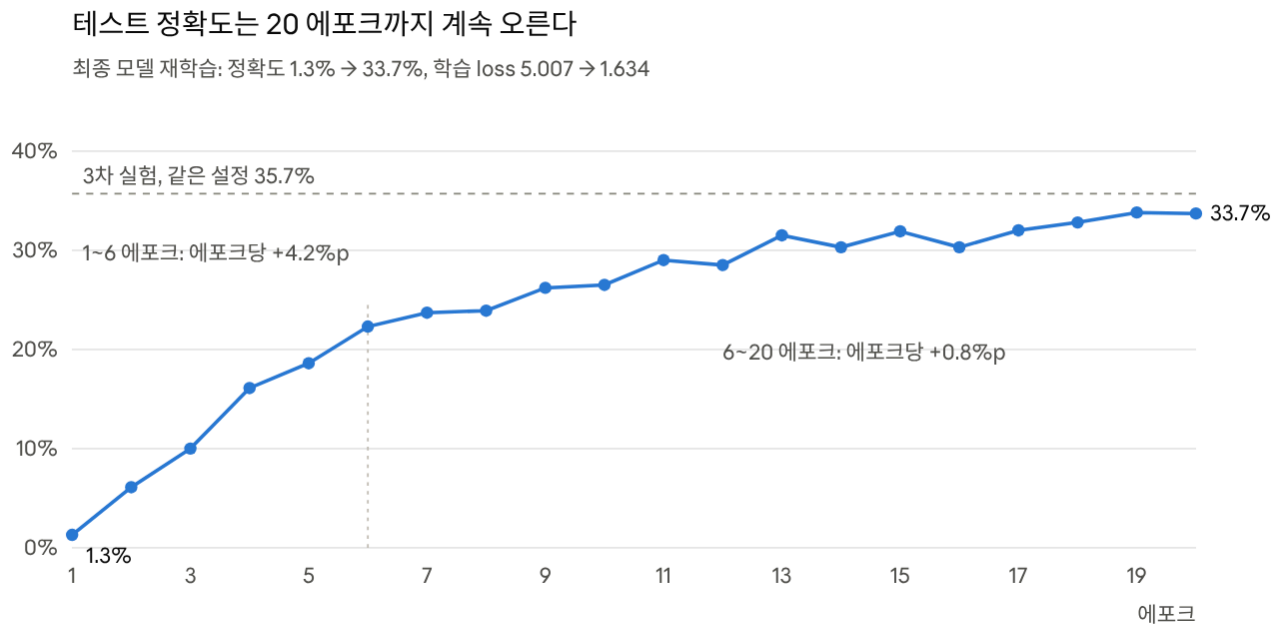
증강	10 에포크 (%)	20 에포크 (%)	20 에포크 학습 시간 (초)
기본	24.2	**35.7**	160
강한	25.5	32.0	401

- **강한 증강은 역전하지 못함.** 20 에포크에서도 기본보다 3.7%p 낮고 2.5배 느렸다. 학습 loss(2.33)가 기본(1.70)보다 높아 더 긴 학습에서는 따라올 여지가 있다.
- **BatchNorm 이득 없음.** 기본 증강, 20 에포크에서 끔 35.7% / 켬 32.4%. 시드 편차 범위 안이라 "이 조건에서 이득이 확인되지 않았다"로 본다.
- **재현성 한계.** 2차와 같은 설정(2층, 기본 증강, 시드 42, 10 에포크)이 27.0%에서 24.2%로 나왔다. GPU 연산의 미세한 무작위성 때문에 시드를 고정해도 완전히 재현되지 않는다.

최종 모델과 결과

최종 설정으로 다시 학습한 결과 테스트 정확도는 33.7%였다. 3차 실험의 35.7%와 합쳐 최종 성능은 **약 34~36% 수준**으로 본다.

항목	최종 값	근거
층 수	2층 (conv 6 → 16, 커널 5×5)	시드 3개 평균 26.3%로 가장 높고 안정적
이미지 크기	224 (원본)	128과 비슷, 64는 크게 하락
배치	16	1차 실험 최고
옵티마이저	Adam, 학습률 0.0001	SGD는 10 에포크에도 거의 못 배움
데이터 증강	기본 (좌우 뒤집기 + 10° 회전)	강한 증강은 낮고 2.5배 느림
Dropout / BatchNorm	없음	이 조건에서 이득 없음
에포크	20	10 → 20에서 +11.5%p



노트북 13번 셀 실행 결과 · 최종 설정 재학습 20 에포크 · 2026-10-16

20 에포크까지 정확도가 꺾이는 구간 없이 오르고, 학습 loss도 매끄럽게 내려간다. 아직 과적합 전이라 에포크를 늘릴 여지가 있다. 에포크마다 1~2%p 오르내리는 것은 테스트 1049장 중 10~20장 차이로, 자연스러운 흔들림이다.

예측 사례. 테스트 사진 8장 중 2장(Rattata, Charmeleon)을 맞혔다. 틀린 6장에는 패턴이 있다.

정답	예측	헷갈린 이유
Golbat (2장)	Zubat	진화 관계, 보라색 박쥐 실루엣
Krabby	Kingler	진화 관계, 주황색 게 모양
Chansey	MrMime	분홍색 둥근 사람형
Flareon	Moltres	주황·불꽃 색감
Goldeen	Spearow	흰색·주황 색 조합

모델은 주로 **색과 대략적인 실루엣**으로 판단한다. 그래서 진화 전후 포켓몬이나 같은 색 계열을 헷갈린다. Golbat과 Krabby는 트레이딩 카드 사진이라 테두리, 글자, HP 표시 같은 무관한 정보도 섞여 있었다.

핵심 교훈

1. **가장 큰 영향은 설정이 아니라 학습량이었다.** 10 → 20 에포크가 +11.5%p로, 어떤 설정 변경보다 컸다. 1·2차의 비교는 모두 덜 학습된 상태에서 이뤄졌다.
2. **3~5%p 이내 차이는 믿기 어렵다.** 시드만으로 최대 6.4%p, GPU 무작위성만으로 약 2~3%p 흔들렸다. 한 번 돌린 결과로 결론 내면 위험하다.
3. **과적합 대책은 충분히 학습할 수 있을 때 효과가 난다.** Dropout과 강한 증강은 짧은 학습에서 학습을 늦추기만 했다.
4. **GPU를 써도 데이터 불러오기가 느리면 속도가 오르지 않는다.** 이미지 크기, num_workers를 바꿔도 학습 시간은 거의 그대로였고, CPU에서 처리하는 강한 증강은 2.5배 느렸다.
5. **효율과 정확도는 다른 문제다.** 4층은 정확도가 약간 낮지만 모델 크기가 1/6이라, 로봇 같은 작은 기기에 넣는다면 더 나은 선택일 수 있다.

한계점

- **테스트 세트로 설정을 골랐다.** 정석은 학습 / 검증 / 테스트 3분할로, 설정은 검증 세트로 고르고 테스트는 마지막에 한 번만 쓰는 것이다. 지금의 정확도는 실제보다 약간 낙관적일 수 있다.
- **대부분의 비교가 시드 1개, 10 에포크다.** 시드 검증은 층 수에만 적용했다.
- **에포크는 20까지만 확인했다.** 20 에포크에서도 정확도가 계속 오르고 있어, 최적 에포크는 아직 모른다.
- **단계별 탐색은 변수 간 상호작용을 놓친다.** 예를 들어 학습률은 배치 32에서 골랐고, 이후 배치를 16으로 바꿨을 때 최적 학습률이 달라졌을 수 있다.
- **데이터 스타일이 제각각이다.** 공식 일러스트, 팬아트, 트레이딩 카드(테두리, 글자, HP 표시 포함)가 섞여 있고 종류당 사진은 평균 35장으로 적다.

- **에포크마다 분할을 바꾸는 방식은 쓸 수 없다.** 검토해 보니 테스트 사진이 학습에 섞여 들어가 정확도가 부풀려진다. 같은 효과는 데이터 증강이나 K-겹 교차검증으로 얻어야 한다.

개선 방향

사전학습 모델 없이 지금 구조 안에서 할 수 있는 개선을 기대 효과 순으로 정리했다.

1. **데이터를 GPU 메모리에 미리 올리기.** 전체 사진을 한 번만 텐서로 바꿔 올려두면(224 크기 uint8 기준 약 0.8GB) CPU 병목이 사라진다. 같은 시간에 에포크를 몇 배 더 돌릴 수 있고, 가장 효과가 컸던 학습량을 늘리는 길이다.
2. **fc1 대신 Global Average Pooling.** 파라미터 542만 개 중 약 99%가 fc1 한 층이다. 채널별 평균으로 줄이면 과적합과 학습 시간이 함께 준다.
3. **필터 늘리고 커널 줄이기.** 필터 6·16개는 150종에 부족하다. 32 → 64 → 128, 3×3 커널로 바꾸고 BatchNorm을 함께 쓴다. 예측 사례에서 본 진화 전후 포켓몬 혼동을 줄이는 데 필요하다.
4. **학습률 스케줄러와 조기 종료.** 0.001에서 시작해 점점 줄이면 초반엔 빨리, 후반엔 세밀하게 배운다. 정확도가 정체되면 자동으로 멈춘다.
5. **평가 방법 개선.** 검증 세트를 따로 두고, 중요한 비교는 시드 2~3개 평균으로 판단한다.
6. **강한 증강 재시도.** 1~4번으로 충분히 학습할 수 있게 된 뒤 다시 비교한다.